TEMCO LEARN

T3000 API & Protocol

System & Utilities API · built Nov 13, 2026

System & Utilities API

Overview

The System & Utilities API provides authentication, health monitoring, file uploads, and system-level operations.

Common Operations

Health Check

TYPESCRIPT

```typescript
const health = await fetch('http://localhost:9103/api/health')
  .then(res => res.json());

console.log(health);
// {
//   status: "healthy",
//   version: "1.0.0",
//   uptime: 86400
// }
```

Login

TYPESCRIPT

```typescript
const response = await fetch('http://localhost:9103/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'admin',
    password: 'password'
  })
});
```

Upload File

```typescript
const formData = new FormData();
formData.append('file', fileInput.files[0]);

await fetch('http://localhost:9103/api/upload', {
  method: 'POST',
  body: formData
});
```

Health & Status Endpoints

GET /api/health

Description: System health check

Request:

```http
GET /api/health HTTP/1.1
```

Response:

```json
{
  "status": "healthy",
  "version": "1.0.0",
  "uptime": 86400,
  "database": "connected",
  "devices_online": 12
}
```

Used By: SystemStatus.tsx

Handler: `api/src/server.rs::health_check`

GET /api/version

Description: Get API version

Request:

```HTTP
GET /api/version HTTP/1.1
```

Response:

```JSON
{
  "version": "1.0.0",
  "build": "20260109",
  "rust_version": "1.75.0"
}
```

Authentication Endpoints

POST /api/login

Description: User login

Request:

```HTTP
POST /api/login HTTP/1.1
Content-Type: application/json

{
  "username": "admin",
  "password": "password"
}
```

Response:

JSON

```
{
  "success": true,
  "user": {
    "id": 1,
    "username": "admin",
    "access_level": 255
  },
  "session_token": "abc123..."
}
```

Used By: LoginPage.tsx

Handler: `api/src/auth.rs::login`

Errors:

- `401` - Invalid credentials

POST /api/logout

Description: User logout

Request:

HTTP

```
POST /api/logout HTTP/1.1
```

Response:

JSON

```
{
  "success": true
}
```

Used By: LogoutButton.tsx

GET /api/session

Description: Check current session

Request:

```http
GET /api/session HTTP/1.1
```

Response:

```json
{
  "authenticated": true,
  "user": {
    "id": 1,
    "username": "admin"
  }
}
```

Used By: AuthProvider.tsx

File Upload Endpoints

POST /api/upload

Description: Upload file

Request:

```http
POST /api/upload HTTP/1.1
Content-Type: multipart/form-data

------boundary
Content-Disposition: form-data; name="file"; filename="config.json"

{...file content...}
------boundary--
```

Response:

```JSON
{
  "success": true,
  "filename": "config.json",
  "path": "/uploads/config.json",
  "size": 2048
}
```

Used By: FileUpload.tsx

Handler: `api/src/file/routes.rs::upload_file`

Errors:

- `400` - No file provided
- `413` - File too large (max 10MB)

GET /api/uploads/:filename

Description: Download uploaded file

Parameters:

- `filename` (path, string) - Filename

Request:

```HTTP
GET /api/uploads/config.json HTTP/1.1
```

Response: File contents with appropriate Content-Type

Used By: FileList.tsx

DELETE /api/uploads/:filename

Description: Delete uploaded file

Parameters:

- `filename` (path, string) - Filename

Request:

```HTTP
DELETE /api/uploads/config.json HTTP/1.1
```

Response:

```HTTP
HTTP/1.1 204 No Content
```

Modbus Endpoints

GET /api/modbus/registers

Description: Read Modbus registers

Parameters:

- `device_id` (query, integer) - Device ID
- `start` (query, integer) - Start register
- `count` (query, integer) - Register count

Request:

```HTTP
GET /api/modbus/registers?device_id=1&start=1000&count=10 HTTP/1.1
```

Response:

```JSON
{
  "device_id": 1,
  "start": 1000,
  "values": [72, 65, 100, 50, 0, 1, 255, 128, 64, 32]
}
```

Used By: ModbusDebugger.tsx

Handler: `api/src/modbus_register/routes.rs::read_registers`

POST /api/modbus/write

Description: Write Modbus register

Request:

```HTTP
POST /api/modbus/write HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "register": 1000,
  "value": 75
}
```

Response:

```JSON
{
  "success": true
}
```

Errors:

- `503` - Device offline or Modbus error

FFI Bridge Endpoints

POST /api/ffi/call

Description: Call BACnet FFI function

Request:

```HTTP
POST /api/ffi/call HTTP/1.1
Content-Type: application/json

{
  "function": "read_property",
  "device_id": 123456,
  "object_type": "analog_input",
  "object_instance": 0,
  "property": "present_value"
}
```

Response:

```JSON
{
  "success": true,
  "value": 72.5
}
```

Used By: BACnetDebugger.tsx

Handler: `api/src/t3_shell/routes.rs::call_ffi_function`

WebSocket Connection

WS /ws

Description: WebSocket for real-time updates

Connection:

TYPESCRIPT

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  console.log('WebSocket connected');
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  console.log('Received:', message);
};

// Send message
ws.send(JSON.stringify({
  action: 1,  // SELECT_PANEL
  panelId: 5,
  serialNumber: 123456
}));
```

Message Types:

- `1` - SELECT_PANEL: Select active device
- `2` - UPDATE_POINT: Point value updated
- `3` - DEVICE_STATUS: Device online/offline
- `4` - REFRESH_COMPLETE: Refresh operation done

Used By:

- WebSocketProvider.tsx
- useWebSocket.ts

Handler: `api/src/t3_socket/handler.rs::handle_websocket`

Implementation Details

Session Management

Sessions are stored in memory with 24-hour expiration:

RUST

```rust
pub struct SessionManager {
    sessions: Arc<RwLock<HashMap<String, Session>>>,
}

pub struct Session {
    user_id: i32,
    username: String,
    created_at: DateTime<Utc>,
    expires_at: DateTime<Utc>,
}
```

File Upload Limits

- Max file size: 10MB
- Allowed types: All (no restrictions)
- Upload directory: `/uploads`

CORS Configuration

RUST

```rust
let cors = CorsLayer::new()
    .allow_origin(Any)
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
    .allow_headers([header::CONTENT_TYPE]);
```

WebSocket Protocol

Messages are JSON-encoded:

TYPESCRIPT

```typescript
interface WebSocketMessage {
  action: number;
  panelId?: number;
  serialNumber?: number;
  data?: any;
}
```

TypeScript Types

TYPESCRIPT

```typescript
export interface HealthStatus {
  status: 'healthy' | 'degraded' | 'unhealthy';
  version: string;
  uptime: number;
  database: string;
  devices_online: number;
}

export interface LoginResponse {
  success: boolean;
  user: {
    id: number;
    username: string;
    access_level: number;
  };
  session_token: string;
}

export interface UploadResponse {
  success: boolean;
  filename: string;
  path: string;
  size: number;
}
```

Error Codes

- `UNAUTHORIZED` - Authentication required
- `INVALID_CREDENTIALS` - Wrong username/password
- `SESSION_EXPIRED` - Session timeout
- `FILE_TOO_LARGE` - Upload exceeds 10MB
- `DEVICE_OFFLINE` - Cannot reach device via Modbus/BACnet